

cmarker.typ

<https://github.com/SabrinaJewson/cmarker.typ>

This package enables you to write CommonMark Markdown, and import it directly into Typst.

simple.typ	simple.md
<pre>#import "@preview/cmarker:0.1.8" #cmarker.render(read("simple.md"))</pre>	<pre># We can write Markdown! *Using* __lots__ ~of~ `fancy` [features] (https://example.org/).</pre>
simple.pdf	
We can write Markdown! <i>Using lots of fancy features.</i>	

This document is available in [Markdown](#) and [rendered PDF](#) formats.

1. Contents

1. [API](#)
2. [Resolving paths correctly](#)
3. [References, labels, figures and citations](#)
4. [Supported Markdown syntax](#)
5. [Interleaving Markdown and Typst](#)
6. [FAQ](#)
7. [Development](#)

2. API

We offer two functions:

1. [render](#)
2. [render-with-metadata](#)

2.1. render

```
render(  
  markdown,  
  smart-punctuation: true,  
  math: none,  
  h1-level: 1,  
  raw-typst: true,  
  html: (:),  
  label-prefix: "",  
  prefix-label-uses: true,  
  heading-labels: "github",  
  scope: (:),  
  show-source: false,  
  blockquote: none,  
) -> content
```

The render function renders the given Markdown and returns content.

2.1.1. markdown

The [CommonMark](#) Markdown string to be processed. Parsed with the [pulldown-cmark](#) Rust library. You can set this to `read("somefile.md")` to import an external markdown file; see the [documentation for the read function](#).

- Accepted values: Strings and raw text code blocks.
- Required.

2.1.2. smart-punctuation

Whether to automatically convert ASCII punctuation to Unicode equivalents:

- nondirectional quotations (" and ') become directional (“ and ”);
- three consecutive full stops (...) become ellipses (...);
- two and three consecutive hyphen-minus signs (-- and ---) become en and em dashes (– and —).

Note that although Typst also offers this functionality, this conversion is done through the Markdown parser rather than Typst.

- Accepted values: Booleans.
- Default value: `true`.

2.1.3. math

A callback to be used when equations are encountered in the Markdown, or `none` if it should be treated as normal text. Because Typst does not support LaTeX equations natively, the user must configure this.

- Accepted values: Functions that take a boolean argument named `block` and a positional string argument (often, the `mitex` function from [the mitex package](#)), or `none`.
- Default value: `none`.

For example, to render math equation as a Typst math block, one can use:

```
#import "@preview/mitex:0.2.6": mitex
#cmarker.render(`$\int_1^2 x \mathrm{d}x$`, math: mitex)
```

which renders as: $\int_1^2 x \, dx$

2.1.4. h1-level

The level that top-level headings in Markdown should get in Typst. When set to zero, top-level headings are treated as titles, `##` headings become `=` headings, `###` headings become `==` headings, et cetera; when set to 2, `#` headings become `==` headings, `##` headings become `===` headings, et cetera.

- Accepted values: Integers in the range `[-128, 127]`.
- Default value: `1`.

2.1.5. raw-typst

Whether to allow raw Typst code to be injected into the document via HTML comments. If disabled, the comments will act as regular HTML comments.

- Accepted values: Booleans.
- Default value: `true`.

For example, when this is enabled, `<!--raw-typst #circle(radius: 10pt) -->` will result in a circle in the document (but only when rendered through Typst). See also [<!--typst-begin-exclude-->](#) and [<!--typst-end-exclude-->](#), which is the inverse of this.

2.1.6. html

The dictionary of HTML elements that cmarker will support.

- Accepted values: Dictionaries whose keys are the tag name (without the surrounding <>) and whose values can be:
 - ("normal", (attrs, body) => [/ * ... * /]): Defines a normal element, where attrs is a dictionary of strings, body is content, and the function returns content.
 - ("void", (attrs) => [/ * ... * /]): Defines a void element (e.g.
, , <hr>).
 - ("raw-text", (attrs, body) => [/ * ... * /]): Defines a raw text element (e.g. <script>, <style>), where body is a string.
 - ("escapable-raw-text", (attrs, body) => [/ * ... * /]): Defines an escapable raw text element (e.g. <textarea>), where body is a string.
 - (attrs, body) => [/ * ... * /]: Shorthand for ("normal", (attrs, body) => [/ * ... * /]).
- Default value: (:).
- Overridable keys: The following HTML elements are provided by default, but you are free to override them: <sub>, <sup>, <mark>, <h1>–<h6>, , , , <dl>, <dt>, <dd>, <table>, <thead>, <tfoot>, <tr>, <th>, <td>, <hr>, <a>, , , <s>, <code>,
, <blockquote>, <figure>, <figcaption>, <svg>, .

For example, the following code would allow you to write <circle radius="25"> to render a 25pt circle:

```
#cmarker.render(read("input.md"), html: (  
  circle: ("void", attrs => circle(radius: int(attrs.radius) * 1pt))  
))
```

2.1.7. label-prefix

If present, any labels autogenerated by footnotes and headings will be prefixed by this string. This is useful to avoid collisions.

- Accepted values: A valid label or the empty string.
- Default value: The empty string.

2.1.8. prefix-label-uses

Whether references to labels such as @label and [link] (#label) should be prefixed by label-prefix.

- Accepted values: Booleans.
- Default value: true.

2.1.9. heading-labels

How the cases of labels autogenerated by headings should be transformed. The label prefix (given in label-prefix) will **not** be transformed.

- Accepted values:
 - "github": Match [GitHub's slugification algorithm](#): strip out invalid characters, lowercase everything, convert ASCII spaces to hyphens, and number duplicate headings to avoid collisions. For example, # My heading! will become <my-heading>.
 - "jupyter": Match [Jupyter's slugification algorithm](#): just convert ASCII spaces to hyphens. For example, # My heading! will become <My-heading!>. Unlike the real Jupyter, we also number duplicate headings, but this shouldn't have any difference in practice since Jupyter doesn't allow referring to duplicate headings anyway.
- Default value: "github".

2.1.10. scope

A dictionary providing the context in which the evaluated Typst code runs. It is useful to pass values in to code inside `<!-- raw-typst -->` blocks, but can also be used to override element functions generated by cmarker itself.

- Accepted values: Any dictionary.
- Default value: `(:)`.
- Overridable keys:
 - All built-in Typst functions.
 - `rule`: Expected to be a function returning content. Will be used when thematic breaks (`---` in Markdown) are encountered. Defaults to `line.with(length: 100%)`.

2.1.11. show-source

A debugging tool. When set to `true`, the Typst code that would otherwise have been displayed will be instead rendered in a code block.

- Accepted values: Booleans.
- Default value: `false`.

2.1.12. blockquote

Deprecated! This used to control how blockquotes were rendered, but we now default to `quote(block: true)`. If you want to override how blockquotes look, either use `#show quote.where(block: true)` or use `scope: (quote: ...)`.

- Accepted values: Functions accepting content and returning content, or `none`.
- Default value: `none`.

2.2. render-with-metadata

```
render-with-metadata(  
  markdown,  
  smart-punctuation: true,  
  math: none,  
  h1-level: 1,  
  raw-typst: true,  
  html: (:),  
  label-prefix: "",  
  prefix-label-uses: true,  
  heading-labels: "github",  
  scope: (:),  
  show-source: false,  
  blockquote: none,  
  metadata-block: none,  
) -> (metadata, content)
```

The `render-with-metadata` functions works the same as `render` with two exceptions:

1. This function returns `(meta, body)`. This allows the user to freely manipulate the metadata.

```
#let (meta, body) = render-with-metadata(input)  
// `body` will be the same as:  
#let body = render(input)
```

2. This function has an extra argument: `metadata-block`. The other arguments are the same as `render`.

2.2.1. metadata-block

How to parse the metadata block. Only a single metadata block at the beginning of the document is supported; metadata blocks not at the start will be ignored. If none, the metadata block will not be parsed. The behaviour is the same as using `render`.

- Accepted values:
 - "frontmatter-raw": Parse the metadata block and return it as a string.
 - "frontmatter-yaml": Parsed the metadata block as YAML and return it as a dictionary.
 - none.
- Default value: none.

3. Resolving paths correctly

Because of how Typst handles paths, elements like images will by default resolve relative to the project root of `cmarker` itself and not your project.

To fix this, one can override the `image` function in the scope the Typst code is evaluated.

```
#import "@preview/cmarker:0.1.8"

#cmarker.render(
  read("yourfile.md"),
  scope: (image: (source, ..args) => image(source, ..args))
)
```

4. References, labels, figures and citations

`cmarker.typ` integrates well with Typst's native references and labels. Where in Typst you would write `@foo`, in Markdown you write `[@foo]`; where in Typst you would write `@foo[Chapter]`, in Markdown you'd write `[Chapter]@foo`. You can also write `[some text](#foo)` to have "some text" be a link that points at `foo`. Headings are automatically given references according to their name, but lowercased and with spaces replaced by hyphens (although this can be controlled with the [heading-labels](#) option):

```
# My nice heading
```

We can make a link to `[this section](#my-nice-heading)`.

We can refer you to `[@my-nice-heading]`. `<!-- generates "We can refer you to Section 1." -->`

Or to `[Chapter]@my-nice-heading]`. `<!-- generates "We can refer you to Chapter 1." -->`

If you have two headings with the same title, they'll be numbered sequentially in the fashion of GitHub Markdown: `heading`, `heading-1`, `heading-2`, etc.

If you want to cite something from a bibliography, you can do it much the same way: `[@citation]`.

If you want to customize the label of a heading from the default, you can use `id=` in HTML, or use `raw-typst` blocks (note that these **must** be preceded by a newline):

```
<h1 id="my-nice-section">My nice heading</h1>
```

```
# My other nice heading
```

```
<!--raw-typst <my-other-nice-section> -->
```

See `[@my-nice-section]` and `[@my-other-nice-section]`.

`<!-- generates "See Section 1 and Section 2." -->`

This can also be used with figures, which is very powerful:

Please refer to `[@my-graph]`. `<!-- generates "Please refer to Figure 1." -->`

```
<figure id="my-graph">
```

```
! [a graph] (graph.png) </figure>
```

If you encounter collision errors across multiple Markdown files, you can set [the label-prefix option](#). For example, setting `label-prefix: "file-a-"` will convert the label of `# My Heading` into `file-a-my-heading`. By default, references within the file will expect the unprefix name; this can be changed by setting [prefix-label-uses: false](#).

5. Supported Markdown syntax

We support CommonMark with a couple extensions.

5.1. Line breaks

A single newline becomes a space; two consecutive newlines (i.e. one blank line) give a paragraph break; two spaces before a newline gives a hard line break (used for example in poetry) – you can also write `<br>` to the same effect.

5.2. Formatting

- *Emphasis*: `*emphasis*` or `_emphasis_`
- **Strong**: `**strong**` or `__strong__`
- ~~Strikethrough~~: `~strikethrough~`
- Subscript: `<sub>subscript</sub>`
- Superscript: `<sup>superscript</sup>`
- **Highlighting**: `<mark>highlighted</mark>`
- Inline code blocks: ``code here``
- Out-of-line code blocks:

```
```\n\ncode here\n```
```

Syntax highlighting can be achieved by specifying a language after the opening backticks:

```
```rust\nlet x = 5;\n```
```

- Blockquotes:

```
> Whereas recognition of the inherent dignity and of\n> the equal and inalienable rights of all members of\n> the human family is the foundation of freedom,\n> justice and peace in the world,
```

5.3. Links

You can link to a URL directly: `[link](https://example.org)`

Or you can reuse URLs by defining them out-of-line:

`[This link]` and `[that link][This link]` go to the same place.

`[This link]: https://example.org`

See [the section on labels](#) for using links with Typst labels and citations.

5.4. Images

```
![alt text here](path/to/image.png)
```

Be aware that this will not work by default – see [Resolving paths correctly](#) for how to fix this.

You can also use the HTML `<img>` tag, which allows you to specify the width and height of the image, either as a % or as an absolute value. The below image will span 50% of the page and be 20pt tall:

```

```

Be aware that image paths that contain spaces must be wrapped in `<>`:

```
<!-- Incorrect: Will render as text -->
```

```
![alt text](image path.png)
```

```
<!-- Correct: Will give an image -->
```

```
![alt text](<image path.png>)
```

Alt text is used by screen readers and other assistive technologies, and won't be visible in the document by default. It should provide a description of the image that makes the document readable to anyone who can't see the image.

If you want to add a caption to the image, you should set a `<figcaption>` explicitly. For example:

```
<figure>
```

```
![alt text](image.png)
```

```
<figcaption>caption text</figcaption>
```

```
</figure>
```

You can alternatively transform all alt text into captions automatically:

```
#cmarker.render(  
  read("example.md"),  
  scope: (  
    image: (source, alt: none, ..args) => {  
      figure(image(source, alt: alt, ..args), caption: alt)  
    },  
  ),  
)
```

However, this has disadvantage that you can't use markup in the caption, because alt text is just a plain string and not content. (In theory, Markdown supports markup in image alt text, but our Markdown parser stringifies it anyway because Typst only supports strings in alt text. It's possible we could add extra functionality for this use case.)

5.5. Headings

Headings are denoted with hashes at the start of the line:

```
# Title of the document
```

```
## Sub-heading
```

```
### Sub-sub-heading
```

The [h1-level](#) option controls how Markdown heading levels become Typst heading levels.

Headings will have Typst labels automatically generated from their content – the `heading-labels` option controls this. If you want to specify a label explicitly, see [the section on labels](#).

5.6. Horizontal rules

Add a horizontal rule with `---`. This corresponds to the Typst code `#rule()`, which, if not overridden by the scope parameter, defaults to `#line(length: 100%)`:

5.7. Lists

Unordered (bullet-point) lists are denoted with `-`, `+` or `*`:

- Foo
- Bar

Ordered (numbered) lists are denoted with `1.` or `1)`:

1. Foo
1. Bar

The numbers will be generated automatically.

5.8. Math equations

Math formulae are denoted with dollars: `$x + y$` or `$$x + y$$`. This requires the [math](#) parameter to be set, and the syntax of the equations is determined by that parameter.

5.9. Tables

Column 1	Column 2
-----	-----
Row 1 Cell 1	Row 1 Cell 2
Row 2 Cell 1	Row 2 Cell 2

You can also use HTML tables, which has greater flexibility for elements inside tables:

```
<table>
  <thead><tr><th>Column 1</th><th>Column 2</th></tr></thead>
  <tr><td>Row 1 Cell 1</td><td>Row 1 Cell 2</td></tr>
  <tr><td>Row 2 Cell 1</td><td>Row 2 Cell 2</td></tr>
</table>
```

5.10. Footnotes

Some text[^footnote]

[^footnote]: content

5.11. Term/description lists

Term/description lists:

```
<dl>
  <dt>Ligature</dt>
  <dd>A merged glyph.</dd>
  <dt>Kerning</dt>
  <dd>A spacing adjustment between two adjacent letters.</dd>
</dl>
```

5.12. Figures

```
<figure><figcaption>My lovely figure</figcaption>
```

```
![a graph](graph.png)
</figure>
```


5.13. Inline SVG

```
<svg version="1.1" width="100" height="100" xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="50" fill="blue" />
</svg>
```

6. Interleaving Markdown and Typst

Sometimes, you might want to render a certain section of the document only when viewed as Markdown, or only when viewed through Typst. To achieve the former, you can simply wrap the section in `<!--typst-begin-exclude-->` and `<!--typst-end-exclude-->`:

```
<!--typst-begin-exclude-->
Hello from not Typst!
<!--typst-end-exclude-->
```

Most Markdown parsers support HTML comments, so from their perspective this is no different to just writing out the Markdown directly; but `cmarker.typ` knows to search for those comments and avoid rendering the content in between.

Note that when the opening comment is followed by the end of an element, `cmarker.typ` will close the block for you. For example:

```
> <!--typst-begin-exclude-->
> One
```

Two

In this code, “Two” will be given no matter where the document is rendered. This is done to prevent us from generating invalid Typst code.

Conversely, one can put Typst code inside a HTML comment of the form `<!--raw-typst [...]-->` to have it evaluated directly as Typst code (but only if [the raw-typst option](#) is set to `true` – which it is by default – otherwise it will just be seen as a regular comment and removed):

```
<!--raw-typst Hello from #text(fill:blue)[Typst]!-->
```

7. FAQ

7.1. How do I include multiple Markdown files in one project?

See [the multi-file example](#).

Note that this example uses a *single* call to render with concatenated Markdown, i.e.

`cmarker.render(read("file-a.md") + read("file-b.md"))`. If you instead wish to use `cmarker.render(read("file-a.md"))` followed by `cmarker.render(read("file-b.md"))`, you may encounter collision issues if two headings have the same name. This can be resolved in two ways:

- Giving the headings an explicit ID: `<h1 id="my-id">My Heading</h1>` instead of `# My Heading`.
- Setting a [label prefix](#), e.g. `label-prefix: "my-file-"`.

7.2. My Markdown after an open HTML tag is getting rendered as text!

This is a Markdown quirk – in code like `<p>hello _world_</p>` or `<!-- -->hello _world_`, italics will not be generated.

There are two fixes: either insert some empty inline-level HTML at the start, e.g. `<span></span><p>hello _world_</p>`, or insert two newlines after the opening tag:

<p>

hello _world_</p>

7.3. How do I centre tables? How do I have them span the page width?

This can be achieved via [show rules](#). For example, centring:

```
#show table: t => align(center, t)
#cmarker.render("
  | a | b |
  | - | - |
  | c | d |
")
```

And having them span the page width:

```
#show table: t => {
  if t.columns.all(c => c == auto) {
    table(columns: (1fr,) * t.columns.len(), align: t.align, ..t.children)
  } else {
    t
  }
}
```

8. Development

File structure is as follows:

lib.typ	The entrypoint of the library, defining its public API. This file only handles the Typst parts; the computation itself is done by the Rust plugin at plugin/.
plugin/	The Rust plugin that is compiled to WebAssembly. This is mostly serialization/deserialization scaffolding around the real logic, which lives in lib/.
lib/	The cmarker-typst pure Rust library, which does the grunt work of the translation itself.
tests/	The test suite. Each Markdown or Typst file here is rendered and compared with the expected HTML output. Run all the tests with <code>cargo test --workspace</code> .
test-runner/	A helper crate that runs our test suite. Consult <code>test-runner/main.rs</code> for a guide on writing tests.
fuzz/	Fuzzing for the library. Run fuzzing with <code>cargo +nightly fuzz run fuzz</code> .
examples/	A few examples. Compile them with <code>typst compile examples/{name}.typ</code> .
prepare-release.sh	A small shell script that can be used to cut a release of cmarker.typ.